

Path_Validation_State

Methodology, Workflow, and Kinematics Report

Simulation-ControlGP — Control/Path_Validation_State

github.com/Moallam456/Simulation-ControlGP/tree/main/Control/Path_Validation_State

1. Executive Summary

Path_Validation_State is an offline, portable C11 state machine that takes an immutable, CRC-locked robot program (a TaughtProgram produced by the Teaching state) and turns it into a fixed-rate, 1 ms Cyclic Synchronous Position (CSP) trajectory that has been independently proven safe to execute before any motor moves. It is the “prove it before you run it” stage of the pipeline: Teaching records the geometry, Path Validation regenerates that geometry deterministically, times it with a jerk-limited S-curve, solves it to joint space, re-checks it with an independent forward-kinematics pass, quantizes it to the exact drive units that will be commanded, and checks joint limits, velocity, acceleration, continuity, singularity margin and collision — sample by sample — before publishing a compact ValidatedTrajectory artifact that Preview and Welding execute without ever running inverse kinematics again.

The module is a direct, byte-for-byte-traceable C translation of the project's MATLAB reference pipeline (SingleSegmentTest_line/arc/fullcircle.m, sCurveTimeScaling.m, controlFK.m, controlJacobian.m and ADLS_IK.m), kept deliberately free of any STM32 HAL dependency so it can run identically on a desktop host, in unit tests, or on the target MCU. This report explains its internal methodology, its finite-state workflow, how it interfaces with the Teaching, Trajectory, Kinematics, Collision, Storage, HMI, Safety and Preview/Welding parts of the system, and the kinematic functions (forward kinematics, Jacobian, numerical inverse kinematics) that live alongside it in the same folder.

2. Where This Module Sits in the Repository

The repository is organized as a MATLAB reference/analysis layer (top-level +analysis, +config, +environment, +robotmodel, +validation packages and Final_Architecture) alongside an embedded-target C layer under Control/, which contains three cooperating state modules and a shared kinematics package:

Folder	Role
Control/Teaching_State	Editable state that records LINE / ARC / CIRCLE segments from an operator/teach-pendant and freezes them into a TaughtProgram.
Control/Path_Validation_State	This module. Consumes the frozen TaughtProgram, regenerates and times the path, solves and verifies it, and publishes a ValidatedTrajectory.
Control/Trajectory	MATLAB reference pipeline for geometry generation and time scaling (LINE/ARC/CIRCLE waypoints, S-curve/trapezoidal/cubic/quintic scaling) that Path_Validation_State's C code is a translation of.
Control/Kinematics	MATLAB reference kinematics (controlFK.m, controlJacobian.m, ADLS_IK.m, DH utilities) that robot_kinematics.c is a translation of.

The Path_Validation_State folder itself contains:

File	Contents
path_validation_types.h	Owned state, configuration, artifact, callback, result and error types.
path_validation.h	Public API: Init, Start, Step, Cancel, CRC helper, report/phase/error name lookups.
path_validation.c	LINE/ARC/CIRCLE geometry, S-curve + orientation generation, the phase orchestration and every safety check.

File	Contents
path_validation_debug.h	Compile-time diagnostic logging macros.
path_validation_test.c	Host unit/integration test with mocked IK/FK/collision services.
path_validation_integration_example.c	Disabled (#if 0) STM32 task sketch showing integration points.
robot_kinematics.h / .c	Forward kinematics, geometric Jacobian, numerical ADLS inverse kinematics, and the callback adapters that plug straight into PathValidationServices.
robot_kinematics_test.c	FK/Jacobian consistency check and ADLS convergence test.

3. Methodology

Path Validation processes a program one segment at a time and, within a segment, one 1 ms sample at a time. The methodology has eight ingredients, applied in the same order for every sample:

3.1 Immutable, CRC-Gated Input

The module never touches a program that could still be edited. Teaching hands over a TaughtProgram together with the draft_revision and a field-wise CRC-32 (PathValidation_CalculateDraftCrc, the same algorithm Teaching uses to freeze the draft). Before doing any work, PathValidation_Start()/Step() re-derive that CRC over every segment and point field and compare it, and the revision, against what was submitted. Any mismatch — for example, Teaching silently continuing to edit the same buffer — fails immediately with PV_ERR_REVISION_MISMATCH or PV_ERR_DRAFT_CRC_MISMATCH before a single sample is generated.

3.2 Structural Validation

Each segment is checked for the right point count for its type (LINE = 2, ARC/CIRCLE = 3), that every point carries finite, non-degenerate numeric data (position, unit quaternion, joint seed), and that frame_id, tool_id and calibration_version are identical across every point in the program relative to the first point. This prevents a program that mixes points recorded under different calibrations or tool offsets from ever being timed or executed.

3.3 Segment Geometry — LINE / ARC / CIRCLE

Geometry is regenerated deterministically from the taught points, matching the MATLAB pipeline exactly:

- LINE — a straight interpolation between P1 and P2; length is the Euclidean distance.
- ARC — a three-point (start / mid / end) circular arc. The circle's center is solved from the two chord vectors $a = P2 - P1$ and $b = P3 - P1$ via their cross product $n = a \times b$ (n also gives the arc's rotation axis); the signed sweep angle is then found by projecting the mid- and end-points onto an orthonormal in-plane frame ($e1, e2$) built from the axis and testing which rotation direction (CW or CCW) actually passes through the recorded midpoint.
- CIRCLE — the same three-point center/radius/frame solve, but sweeps the full $\pm 2\pi$ using the taught circle_direction (CW/CCW) instead of an inferred end angle.

- Degenerate geometry (collinear points, near-zero cross product, radius under the configured minimum segment length) is rejected with PV_ERR_DEGENERATE_GEOMETRY before any sample is produced.

3.4 Orientation Interpolation

For LINE segments, orientation is a standard SLERP between the start and end quaternions (with a linear-interpolation fallback and re-normalization when the two orientations are almost identical, to avoid a divide-by-near-zero). For ARC/CIRCLE segments, orientation follows the taught rotation itself: a quaternion representing $u \cdot \theta_{\text{total}}$ of the swept rotation is applied to the start orientation, and any residual difference between that prediction and the taught end orientation is SLERPed in on top — so the tool's orientation tracks the arc's own rotation rather than interpolating independently of the path shape.

3.5 Seven-Phase Jerk-Limited S-Curve Timing

Every segment is time-scaled with the same seven-phase (jerk / const-accel / jerk / const-vel / jerk / const-decel / jerk) S-curve used by the MATLAB `sCurveTimeScaling.m`, evaluated on a path normalized to arc length (velocity, acceleration and jerk limits are all divided by the segment length so the curve is solved in a dimensionless $u \in [0, 1]$ progress variable). `build_scurve()` selects among the standard cases — full trapezoid, triangular velocity profile, or a jerk-only “S” when the segment is too short to reach the commanded acceleration or velocity — and produces per-phase duration, jerk, and starting position/velocity/acceleration, which `eval_scurve()` then samples at each 1 ms tick to get normalized progress u and speed.

3.6 Generic Pose-to-Joints Solve and Drive Quantization

The Cartesian target for each sample is handed to a pluggable inverse-kinematics callback (`PathValidationServices.inverse_kinematics`) seeded with the previous sample's joint solution (or the taught seed for the first sample of the program), so consecutive samples solve to the physically closest joint configuration. The state itself has no ADLS dependency — `robot_kinematics.c` supplies the concrete numerical solver, but any solver with the same signature can be substituted. Every accepted IK solution is then converted per-joint to the signed 32-bit CiA 402 CSP target-position unit (object 0x607A) using the commissioned `drive_units_per_joint_rad / drive_zero_offset_units / joint_direction_sign`, and converted back to radians; the difference between the original and the round-tripped value is checked against `maximum_position_quantization_error_rad`. Every later check (FK, limits, derivatives, collision) therefore validates the exact quantized command that will be sent to the drive, not the unquantized floating-point IK result.

3.7 Independent Verification

Each generated/quantized sample is re-checked from multiple independent angles before it is accepted:

- Forward kinematics is re-run on the quantized joint solution and compared against the original Cartesian target; position and orientation error must be under `maximum_fk_position_error_m / maximum_fk_orientation_error_rad`.
- Joint position is checked against `joint_min_rad / joint_max_rad`.
- Joint velocity and (optionally) acceleration are estimated by finite difference against the previous sample and checked against configured maxima.

- Joint discontinuity is checked directly: the step between consecutive samples cannot exceed `maximum_joint_step_rad`.
- The IK solver's smallest Jacobian singular value (`sigma_min`) is checked against `minimum_singularity_sigma`, keeping the whole trajectory away from kinematic singularities.
- An external `collision_free()` callback receives every quantized joint sample before it is written, so environment/self-collision checking stays owned by the collision module rather than being duplicated here.

3.8 CRC-Chained Artifact Output

As samples are accepted they are streamed through an abstract `PathValidationStorage` interface (`begin / write_sample / commit / abort`) so the RAM-backed host test and a future STM32 external-flash backend share the same contract. A running CRC-32 is accumulated over every stored sample; at `FINALIZE`, that sample-data CRC plus the segment index table, counts, duration and path length are assembled into the `ValidatedTrajectory` metadata and covered by a second, artifact-level CRC-32 before `storage.commit()` is called and the result becomes `VALID`.

4. State Machine Workflow

`PathValidation_Step()` is a cooperative, budgeted state machine: each call processes at most `sample_budget` samples and returns control to the caller, so a supervisory task can interleave Path Validation with other periodic work instead of blocking on an entire (potentially long) program in one call. One IK callback invocation is still bounded by the kinematics module's own iteration limit. The phases (`PathValidationPhase`) are:

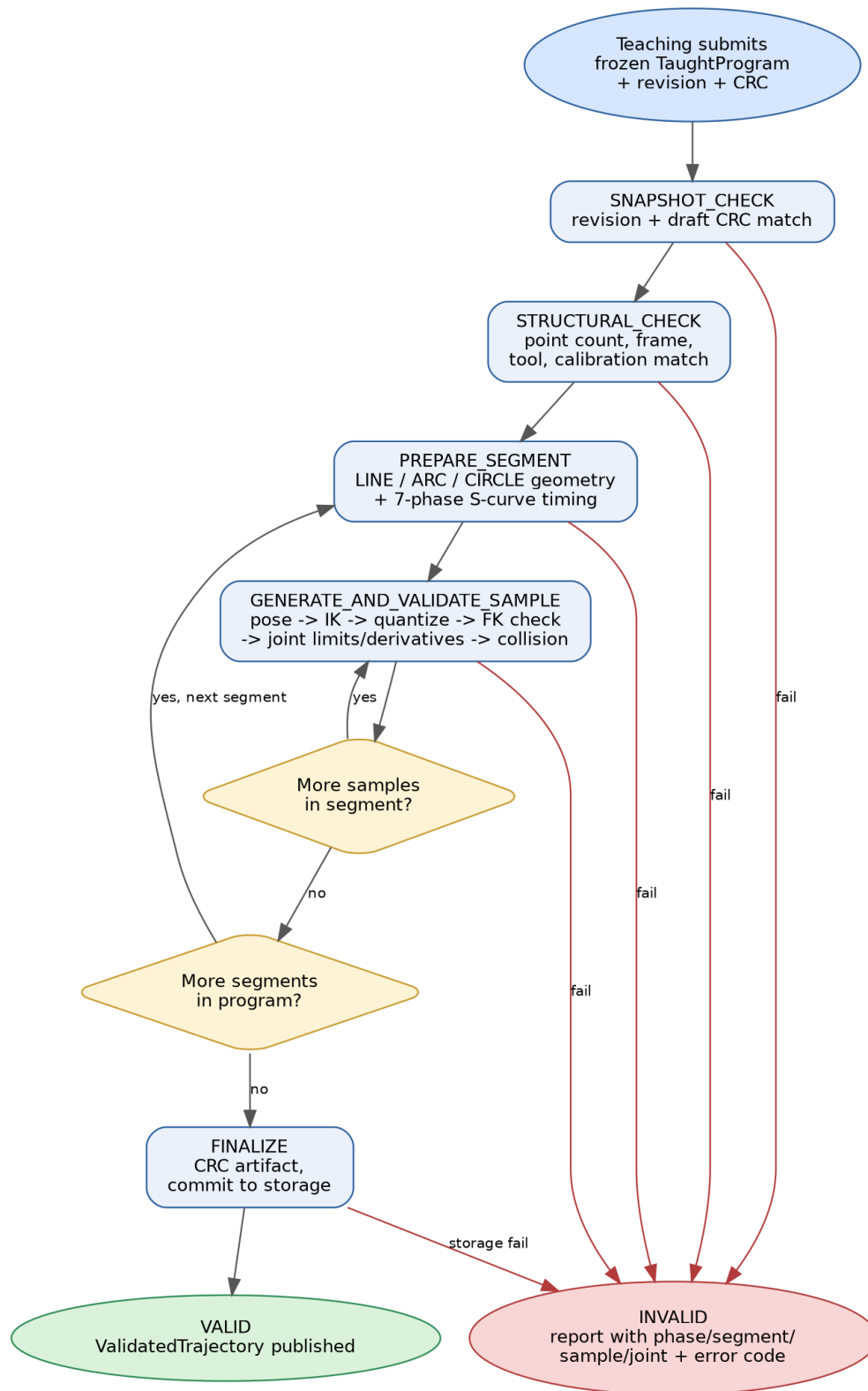


Figure 1 — Path Validation State phase flow, from a frozen TaughtProgram to a published ValidatedTrajectory.

4.1 Phase-by-Phase Detail

Phase	What happens	On failure
IDLE	Waiting for PathValidation_Start(); set by Init and by Cancel.	—
SNAPSHOT_CHECK	Verifies submitted revision and draft CRC against the live TaughtProgram; rejects an empty program.	REVISION_MISMATCH, DRAFT_CRC_MISMATCH, EMPTY_PROGRAM
STRUCTURAL_CHECK	Validates point counts, numeric validity, and frame/tool/calibration consistency across the whole program; seeds the IK chain from the first taught joint pose.	UNSUPPORTED_SEGMENT, INCOMPLETE_SEGMENT, INVALID_POINT, FRAME/TOOL/CALIBRATION_MISMATCH
PREPARE_SEGMENT	Builds LINE/ARC/CIRCLE geometry, resolves the commanded speed, and solves the 7-phase S-curve for the current segment; computes how many 1 ms samples the segment needs.	DEGENERATE_GEOMETRY, ZERO_LENGTH_SEGMENT, INVALID_CIRCLE_DIRECTION, INVALID_PARAMETER
GENERATE_AND_VALIDATE_SAMPLE	For each 1 ms tick: evaluates the S-curve, builds the Cartesian target, solves IK, quantizes to drive units, re-verifies with FK, checks joint limits/velocity/acceleration/continuity and singularity margin, calls the collision callback, and writes the sample to storage.	IK_FAILED, SINGULARITY_MARGIN, POSITION_CONVERSION, FK_POSITION/ORIENTATION, JOINT_POSITION/VELOCITY/ACCELERATION/DISCONTINUITY, COLLISION, STORAGE, SAMPLE_CAPACITY
FINALIZE	Fills in ValidatedTrajectory metadata (duration, path length, segment table), computes the sample-data and artifact CRCs, and commits to storage.	STORAGE
VALID	Terminal success state; report.result = PV_RESULT_VALID.	—
INVALID	Terminal failure state; the report records the failing phase, segment, sample and (where relevant) joint plus a specific PathValidationError.	—

4.2 Sample and Segment Looping

Inside GENERATE_AND_VALIDATE_SAMPLE, the state loops locally: as long as segment_local_sample stays under segment_sample_count it keeps generating the next sample of the same segment. When a segment finishes, its length is added to the artifact's running path_length_m and the phase returns to

PREPARE_SEGMENT for the next segment (local sample 0 is skipped on every segment after the first, because it would duplicate the previous segment's end-point and create a zero-dt pair in the global sample stream). When the last segment finishes, the phase moves to FINALIZE. PathValidation_Cancel() can interrupt a RUNNING validation from outside at any point, aborting storage and moving straight to INVALID with PV_ERR_CANCELLED — this is the hook the Safety/Fault manager uses; Path Validation never clears a global fault or E-stop itself, it only reacts to being cancelled.

5. Kinematic Functions (robot_kinematics.c / .h)

robot_kinematics.c deliberately keeps every trigonometric/matrix computation out of path_validation.c, exposing only the callback shapes (PvInverseKinematicsFn, PvForwardKinematicsFn) that PathValidationServices expects. It is a direct C11 port of the project's MATLAB kinematics, described in the README as: FK = controlFK.m, Jacobian = controlJacobian.m, IK = ADLS_IK.m.

5.1 Forward Kinematics — RobotKinematics_Forward()

Builds the tool-center-point pose from six joint angles using standard Denavit–Hartenberg transforms. For each joint i , `dh_transform(a_i, d_i, alpha_i, q_i + theta_offset_i)` builds the 4×4 homogeneous transform:

```
A_i =  
[ cosθ  -sinθ·cosa  sinθ·sina  a·cosθ ]  
[ sinθ   cosθ·cosa -cosθ·sina  a·sinθ ]  
[  0      sina      cosa      d      ]  
[  0      0         0         1      ]
```

The six per-joint transforms are chained ($T = A_1 \cdot A_2 \cdot \dots \cdot A_6$) and post-multiplied by the configured flange-to-TCP transform (`RobotKinematicsModel.T_flange_tcp`) to reach the tool point. Position is read directly from the translation column; orientation is converted from the resulting 3×3 rotation block to a unit quaternion (`rotation_to_quaternion()`, the standard trace/largest-diagonal-term method with re-normalization).

5.2 Geometric Jacobian — RobotKinematics_Jacobian()

Walks the same DH chain once, recording each joint's origin and its z-axis (rotation axis) in the base frame as it goes, then reuses the final TCP position p computed from the chain. For every revolute joint i , the standard geometric-Jacobian columns are:

```
J_linear_i = z_i × (p - o_i)  
J_angular_i = z_i
```

assembled into a 6×6 matrix in row-major order with linear-velocity rows stacked over angular-velocity rows, matching `controlJacobian.m`. This Jacobian is reused both inside the numerical IK solve and, via its smallest singular value, as the path's singularity margin.

5.3 Numerical Inverse Kinematics — RobotKinematics_AdlsIk()

Despite the .m file's name, `ADLS_IK.m` (and this C port) is not a closed-form/analytical solver — it is an iterative Adaptive Damped Least-Squares (a Levenberg–Marquardt-style damped pseudo-inverse) numerical method, which is why the module classifies it as `ROBOT_IK_NUMERICAL_ADLS` and leaves an explicit, currently empty `ROBOT_IK_ANALYTICAL` slot (`RobotAnalyticalIkFn`) for a future closed-form solver. Each iteration:

- Runs forward kinematics on the current joint guess and computes a Cartesian position error and an orientation error via the $SO(3)$ logarithm of the error rotation (`rotation_log_vector` — handles the near-0 and near- π rotation-angle special cases explicitly).
- Stops and reports convergence once both errors are under `position_tolerance_m` / `orientation_tolerance_rad`.
- Otherwise computes the 6×6 Jacobian and its smallest singular value `sigma_min` via a cyclic Jacobi eigenvalue sweep on $J^T J$ (`sigma_min_6x6()`), and uses it to set an adaptive damping factor λ that grows

smoothly as $\sigma_{\min}$ falls below $\sigma_{\text{threshold}}$: $\lambda = \lambda_{\max} \cdot \sqrt{1 - (\sigma/\sigma_{\text{threshold}})^2}$ — i.e. undamped away from singularities, increasingly damped near them.

- Solves the damped normal equations $(J^T J + \lambda^2 I) \cdot \Delta q = J^T \cdot \text{err}$ with Gaussian elimination and partial pivoting (`solve6()`), scales the step by `step_size`, and clamps the result to `joint_min_rad/joint_max_rad` every iteration.
- Runs for at most `max_iterations`, always returning the best joint solution found together with a `PvIkDiagnostics` record (converged, iterations, position/orientation error, $\sigma_{\min}$) that Path Validation uses directly for its own singularity-margin and IK-failure checks.

The README is explicit that ADLS is intended for offline Path Validation only, not for the 1 ms CSP interrupt itself — its worst-case execution time on the target STM32 core must be measured and its iteration count bounded before commissioning.

5.4 Callback Adapters

`RobotKinematics_PathIk()` and `RobotKinematics_PathFk()` adapt the model-level functions to the exact function pointer signatures `PathValidationServices` expects, and `RobotKinematics_MakePathValidationServices()` assembles a ready-to-use `PathValidationServices` struct (IK + FK + an injected collision callback) from a `RobotKinematicsContext`. `RobotKinematicsContext.selected_method` chooses at runtime between the numerical ADLS path and a future analytical solver, so a closed-form IK can be dropped in later without touching Path Validation.

6. Data Ownership and Parameter Sourcing

Path Validation is intentionally a thin orchestrator: almost everything it touches is owned elsewhere and reaches it as either a read-only snapshot or a callback.

Data	Owner	Path Validation's access
Frozen TaughtProgram	Program / Teaching manager	Read-only
Revision and draft CRC	Program manager	Verify
Validation phase / cursors / report	Path Validation	Read/write
S-curve configuration and thresholds	Commissioned configuration	Snapshot / read
Pose-to-joints solver, FK, robot model	Kinematics module	Callback
Environment collision model	Collision module	Callback
CSP sample stream	Program Store / external-memory driver	Write through callback
ValidatedTrajectory metadata	Path Validation until success; then Program Store	Write / publish
HMI progress / messages	HMI module	Receives report
Global faults / E-stop	Safety / Fault manager	Can cancel; never cleared here

The README also stresses that no motion parameter is embedded as a placeholder constant: TCP speed comes from the recorded segment/program recipe, TCP acceleration/jerk limits and FK error thresholds from commissioned motion configuration, the sample period is a fixed 1000 μ s contract, joint limits come from the final robot/drive configuration, and `drive_units_per_joint_rad / drive_zero_offset_units / joint_direction_sign` must be derived from the selected encoder resolution, electronic gearing, gearbox ratio and homing convention — the values used in `robot_kinematics_test.c` are explicitly a UR5-scale test fixture, not production constants.

7. Crossovers With Other States and Modules

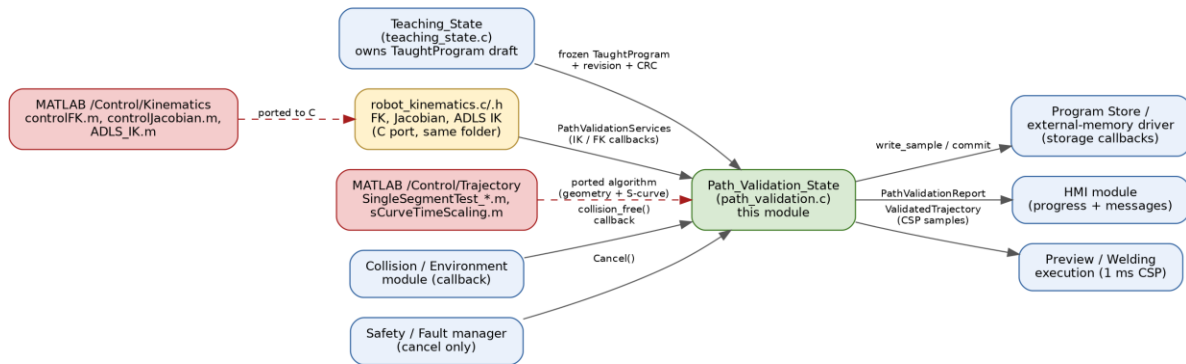


Figure 2 — `Path_Validation_State`'s interfaces to the Teaching state, the MATLAB reference pipeline it was translated from, and the surrounding STM32 modules.

7.1 Teaching_State — upstream producer

`Teaching_State` owns the editable draft: segment selection, point recording, speed adjustment and reset all happen there, with no MATLAB dependency and no HAL calls. Its README is explicit that “Path Validation — not Teaching — runs geometry generation, arc-length parameterization, time scaling, IK, FK checking, limits and collision checks,” i.e. the two states share the same conceptual per-segment interface (start/end position, orientation, commanded speed, an initial IK seed) but Teaching never performs any of the actual validation math. When `TEACH_EVENT_VALIDATE_PATH` fires, `TeachingOutputs.validation_request` pulses for one cycle with the submitted_program_id/revision/crc that Path Validation's `Start()` call consumes; `TEACH_EVENT_RESET` increments the draft revision precisely so that a stale Path Validation result can never match a newer draft.

7.2 MATLAB Control/Trajectory — algorithmic source of truth

`path_validation.c`'s LINE/ARC/CIRCLE geometry and its seven-phase S-curve timing are, per the module's own source mapping, translations of `SingleSegmentTest_line.m`, `SingleSegmentTest_arc.m`, `SingleSegmentTest_fullcircle.m` and `sCurveTimeScaling.m` in `Control/Trajectory`. That folder also contains the broader MATLAB trajectory-pipeline reference (arc-length parameterization, trapezoidal/cubic/quintic time scaling, joint-space trajectory generation, and multi-segment Cartesian-pose test harnesses) that the C module's design was validated against; `Trajectory/Pipeline/compare_matlab_c_line.m` and the accompanying CSV pairs exist specifically to numerically cross-check the MATLAB and C line pipelines against each other.

7.3 MATLAB Control/Kinematics — kinematic source of truth

`robot_kinematics.c` is the C translation of `controlFK.m`, `controlJacobian.m` and `ADLS_IK.m` from `Control/Kinematics`. That MATLAB package also contains a symbolic FK/Jacobian pair (`symbolicFK.m`, `symbolicJacobian.m`), a singularity check, SLERP orientation helper, a restart-capable ADLS variant (`ADLS_restart_IK.m`), and later analytical-IK experiments (`Kinematics/NewIK`, `Kinematics/NewNewIK`) aimed at eventually filling the `ROBOT_IK_ANALYTICAL` slot that `robot_kinematics.h` already reserves.

7.4 Collision / Environment module

Path Validation never implements collision checking itself; PvCollisionCheckFn is injected through PathValidationServices and called with the fully quantized joint sample, the owning segment, and the sample index for every accepted pose, mirroring the project's separate +analysis collision/workspace tooling (collisionFreeWorkspace.m, environmentConstrainedWorkspace.m) at the embedded level.

7.5 Program Store / external-memory driver

The PathValidationStorage callback quartet (begin / write_sample / commit / abort) is the only place Path Validation touches persistence. The current host test backs it with RAM; the module notes the eventual STM32 backend should buffer 24-byte CSP samples into flash pages and only commit the metadata/valid marker after the payload CRC succeeds — keeping the crossover boundary identical regardless of the underlying medium.

7.6 HMI, Safety, and Preview/Welding — downstream consumers

PathValidationReport (phase, result, error, failed segment/sample/joint, FK error peaks, singularity margin, peak joint velocity/acceleration) is designed to be read directly by the HMI module for operator feedback. The Safety/Fault manager can call PathValidation_Cancel() at any time but never clears a fault itself — that stays outside this state entirely. On success, the published ValidatedTrajectory (with its CRC-verified, fixed-rate CSP samples) is exactly what Preview and Welding execution consume: they transmit stored samples on the EtherCAT cycle rather than regenerating IK, so all the expensive numerical work happens once, offline, inside this state.

8. Role Within the Overall Robot Control State Machine

Across the three Control/ state modules, the system forms a simple, safety-motivated pipeline:

- Teaching (editable) → records LINE/ARC/CIRCLE segments and freezes a draft with a revision and CRC.
- Path Validation (offline, this module) → regenerates geometry, times it, solves and verifies it sample-by-sample, and publishes an immutable, CRC-sealed ValidatedTrajectory or a detailed INVALID report.
- Preview / Welding (real-time, 1 ms CSP) → replays the already-validated, already-quantized samples with no further IK, staying inside the fixed EtherCAT cycle budget.

This split is what lets the numerically expensive, iteration-bounded ADLS solve and its Jacobi-sweep singular-value computation happen outside the hard real-time interrupt, while still guaranteeing that only geometry that has passed FK re-verification, joint-limit/derivative/continuity checks, a singularity-margin check and a collision check can ever reach the drives. The CRC chain (draft CRC → sample-data CRC → artifact CRC) threads through all three states so that a Preview/Welding task can trust it is executing exactly the trajectory that was validated, not a program that was edited afterward.

9. Key Constraints and Limitations

- LINE requires exactly 2 points; ARC requires start/mid/end; CIRCLE requires three non-collinear points plus an explicit CW/CCW direction.
- The state has no hard dependency on ADLS specifically — the generic IK/FK callbacks must be wired to whichever embedded solver is finally chosen; the host test uses deterministic mocks, not `robot_kinematics.c`, for its own logic tests.
- ADLS is intended for offline Path Validation only; its worst-case execution time and iteration bound on the target STM32 core must be measured before commissioning — it is not meant to run inside the 1 ms CSP interrupt.
- The second numerical derivative (joint acceleration) is sensitive to sample time and IK noise and should be validated against MATLAB reference vectors.
- Each stored sample is exactly 24 bytes (six signed 32-bit CSP target positions); Cartesian pose, speed, velocity and acceleration are validation-time-only data and are not retained in the artifact.
- Example numeric values in the repository (0.10 m/s, 0.25 m/s², 1.0 m/s³, 0.05 s, UR5 joint limits, the current singularity threshold) are explicitly test fixtures and must not be copied into a production configuration without deriving and documenting them for the final arm, servo drives, gearbox ratios and welding process.
- A passed software validation is not a safety function and does not replace the safety controller, protective devices, safe commissioning, or physical preview of the taught program.

10. Testing Approach

`path_validation_test.c` exercises the full state machine end-to-end on a three-segment program (LINE → ARC → CIRCLE) using deterministic mock IK/FK/collision services and a RAM storage backend, and asserts: successful completion with a plausible sample count, duration and path length; correct per-segment sample-index

bookkeeping; that a changed payload submitted under the old CRC is rejected with `PV_ERR_DRAFT_CRC_MISMATCH`; and that degenerate (collinear) ARC geometry is rejected with `PV_ERR_DEGENERATE_GEOMETRY`. `robot_kinematics_test.c` separately checks FK/Jacobian consistency and ADLS convergence in isolation from the state machine. Both are plain host-buildable C (`cc -std=c11 -Wall -Wextra -Werror -pedantic`), so they run identically in CI, on a developer desktop, or as a pre-flash sanity check.

11. Conclusion

`Path_Validation_State` is the verification backbone between an editable taught program and real-time robot motion. Its methodology — CRC-gated input, deterministic LINE/ARC/CIRCLE geometry regeneration, jerk-limited S-curve timing, a pluggable IK solve with drive-unit quantization, independent FK re-verification, joint limit/derivative/continuity/singularity checks, and an external collision callback — is expressed as a small, budgeted, interruption-tolerant finite state machine (`PathValidation_Step()`) that keeps every expensive numerical operation outside the hard-real-time control loop. Its crossovers are equally deliberate: it consumes Teaching's frozen output, reuses (via `robot_kinematics.c`) a direct C port of the project's MATLAB FK/Jacobian/ADLS-`IK` kinematics, delegates collision checking and persistence to their owning modules, and publishes a compact, CRC-sealed `ValidatedTrajectory` that Preview and Welding can execute at a fixed 1 ms EtherCAT cycle without ever solving inverse kinematics again.